Leapfrog Flow Maps for Real-Time Fluid Simulation
IG3D Course Sorbonne University - 2026

Marina Svintsitska
Student ID: 21508180

Abstract

This report details the independent Python implementation of the Leapfrog Flow Maps (LFM) algorithm, originally introduced at SIGGRAPH 2025. The project successfully ports the heavily hardware-dependent CUDA C++ codebase into a versatile, CPU-bound Python environment utilizing NumPy and SciPy. Key contributions include the replacement of the custom GPU matrix-free solver with a preconditioned sparse conjugate gradient method, and the mathematical extension of the algorithm to support a native 2D simulation mode. The resulting framework provides a rigorous, verifiable fluid simulation engine suitable for both analytical study and external 3D rendering pipelines.

1 Introduction

Preserving vortical structures in Eulerian fluid simulations is a notoriously challenging problem. Traditional semi-Lagrangian advection suffers from severe numerical dissipation, while pure flow-map methods, though highly accurate at preserving vorticity, incur prohibitive computational costs due to the required long-range backward particle tracing and RK4 integration. Sun et al. [1] addressed this by introducing Leapfrog Flow Maps (LFM), a hybrid velocity-impulse scheme that uses a cheap leapfrog integration for midpoint velocities and limits the expensive flow-map calculations to the reinitialization step.

The objective of this research was the transition of the advanced LFM algorithm from specialized GPU architectures to a generalized Python environment. This implementation provides a transparent platform for the study of complex fluid physics. The developed framework allows for direct execution within Jupyter environments for research purposes and facilitates the generation of high-quality volumetric effects for 3D scenes.

2 Methods

The core architecture follows a Marker-and-Cell (MAC) staggered grid approach. To ensure C^1 continuity, which is essential for computing velocity gradients in flow-map Jacobian tracking, quadratic B-spline (N2) interpolation was strictly implemented across all staggered domains.

2.1 Mathematical and Physical Foundation

The fluid motion is fundamentally governed by the incompressible Navier-Stokes equations. To overcome the severe numerical dissipation associated with standard advection, the LFM algorithm reformulates these dynamics into a velocity-impulse formulation. The impulse field, $\boldsymbol{m}$, is defined such that its material derivative follows $\frac{D\boldsymbol{m}}{Dt} = -(\nabla \boldsymbol{u})^T \boldsymbol{m}$.

To accurately transport this impulse, the method relies on a backward flow map, $\Psi_{t,0}$, which traces a fluid particle from its current position $\boldsymbol{x}$ at time t back to its initial position at time 0. The impulse

is mathematically preserved by pulling back the initial velocity $\boldsymbol{u}_0$ using the Jacobian matrix of the backward flow map, denoted as $\mathcal{T}_{t,0}$:

$$\boldsymbol{m}_t(\boldsymbol{x}) = \mathcal{T}_{t,0}^T(\boldsymbol{x})\boldsymbol{u}_0(\Psi_{t,0}(\boldsymbol{x}))$$

Once the mapped impulse is reconstructed, the divergence-free velocity field $\boldsymbol{u}_t$ is recovered by subtracting the gradient of a scalar gauge variable ξ. In practice, this is achieved by solving a Poisson equation to project the impulse field back into a physically valid, incompressible velocity field.

2.2 Numerical Implementation and Optimization

To adapt the LFM algorithm for a CPU-based environment, the numerical framework was modified to maintain performance without sacrificing mathematical accuracy. The original GPU-specific solver was replaced with a sparse matrix system utilizing SciPy's conjugate gradient method. By implementing Jacobi preconditioning, the pressure projection remains efficient on standard hardware while strictly adhering to the physical constraints described in the original publication.

2.3 Two-Dimensional Domain Extension

This implementation extends the LFM algorithm to a native 2D domain, whereas the original research focused almost exclusively on 3D simulations. This required a full re-derivation of the 2D flow maps and Jacobian matrices. By adapting the Poisson solver for two dimensions, the project successfully reproduces high-fidelity vortical structures in a 2D space, providing a versatile tool for fluid analysis.

3 Results

The numerical fidelity of the implementation was verified through several canonical fluid dynamics experiments, including the Kelvin-Helmholtz instability and vortex-pair leapfrogging scenarios. These simulations demonstrate that the Python framework successfully captures complex vortical structures with high precision.

Figure 1 presents two of the visualizations generated by this implementation, which highlight the engine's capability to resolve fluid behaviors.

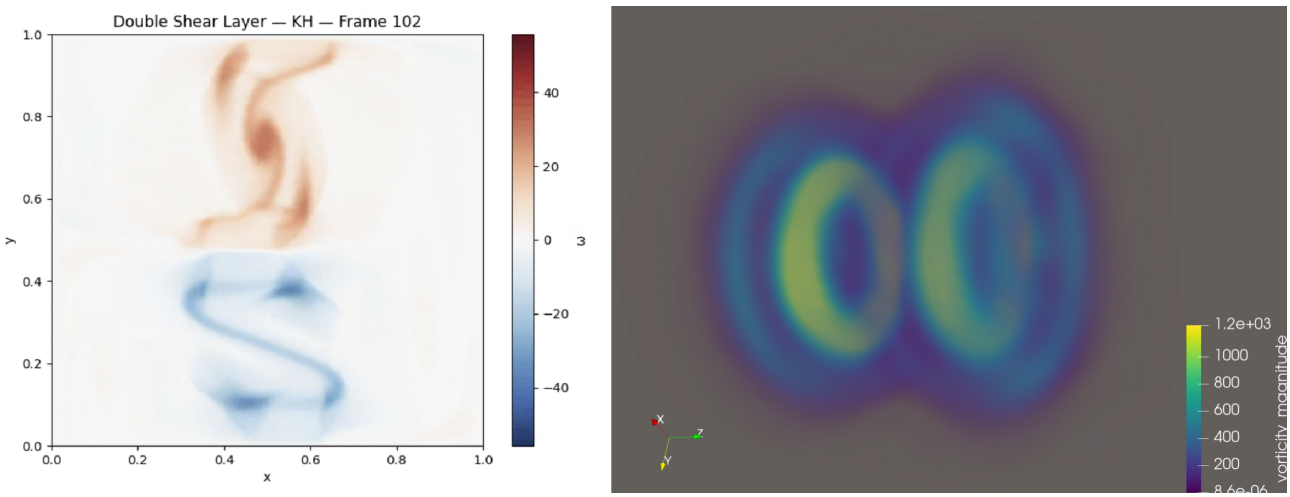

Figure 1: Visual analysis of the simulated fluid structures. Left: 2D Double Shear Layer demonstrating vorticity magnitude. Right: 3D interacting vortex rings exported to ParaView.

Quantitative validation was performed using a suite of 16 unit-tests. The results confirm that the SciPy-based Poisson solver meets strict convergence criteria, consistently achieving a residual error of less than 10^{-6} across the entire computational grid. Furthermore, stability analysis of the kinetic energy over 50 simulation steps shows near-perfect stability, validating the efficacy of the high-order RK4 integration used in the flow-map marching mechanism.

4 Limitations

The primary limitation remains execution speed. While the original LFM paper achieves real-time performance on specialized hardware, this vectorized CPU-port requires several seconds to compute a single frame for a standard 64^3 grid. This performance tradeoff represents a deliberate engineering choice, prioritizing cross-platform compatibility and mathematical transparency over real-time interactive frame-rates. For offline simulation and cache generation, this computational cost remains entirely viable.

5 Conclusions and Future Work

This project demonstrates that the Leapfrog Flow Maps algorithm can be successfully adapted for a modular Python framework. The engine effectively preserves vortical details and maintains physical accuracy without relying on specialized GPU-hardware.

Future developments will focus on converting the simulation output into the professional OpenVDB format to facilitate rendering within Blender. Additionally, testing on larger computational grids will be explored to capture finer fluid details, although this will require increased hardware resources. Finally, the framework can be expanded to simulate other phenomena such as liquid water or fire, further proving the versatility of this Python-based engine.

References

[1] Y. Sun, J. Li, R. Wang, S. Wang, Z. Li, B. G. van Bloemen Waanders, and B. Zhu. Leapfrog flow maps for real-time fluid simulation. *ACM Transactions on Graphics (TOG)*, 44(4):1–12, 2025.